

생성자 역할 설명



GAN모델에서 생성자의 역할은

GAN에서 **생성자(Generator, G)**의 역할은 한 문장으로 요약하면:

“진짜처럼 보이는 가짜 데이터를 만들어서 판별자(D)를 속이는 것.”

아주 쉽게, 중·고급까지 층별로 정리해줄게.

✓ 1. 초초간단 버전

생성자의 목표 =

랜덤 노이즈 → 진짜처럼 보이는 이미지로 변환

즉,

- 입력: $z \sim N(0,1)$ (완전 랜덤 벡터)
- 출력: $G(z)$ = 진짜 같은 이미지
- 목표: $D(G(z))$ 가 1(진짜)라고 판단하도록 속이기

✓ 2. 쉬운 버전 (직관적 이해)

생성자는 화가다.

- 판별자(D)는 감정평가관.
- 생성자(G)는 아무 재료 없이 그림을 그린다.
- 처음엔 형편없지만, 감정평가관(D)에게 혼나면서 점점 실력이 좋아진다.
- 최종 목표:
D가 가짜임을 전혀 눈치 못 채도록 완벽한 그림을 그리는 것

✓ 3. 정식적 정의

생성자 G의 목적 함수

G는 다음을 최소화하려고 한다:

$$\min_G \mathbb{E}_{z \sim p(z)} [\log(1 - D(G(z)))]$$

하지만 실제로는 학습이 더 잘 되는 형태:

$$\max_G \mathbb{E}_z[\log(D(G(z)))]$$

즉,

- $D(G(z))$ 가 높아지도록($G(z)$ 를 진짜처럼 보이게) 학습

✓ 4. 코드 관점에서 정리

🔥 Discriminator 학습

```
python

pred_real = D(real)
loss_real = BCE(pred_real, 1)

pred_fake = D(fake.detach())
loss_fake = BCE(pred_fake, 0)

loss_D = loss_real + loss_fake
```

🔥 Generator 학습

```
python

pred_fake_for_G = D(fake)          # detach 안 함!
loss_G = BCE(pred_fake_for_G, 1)    # D를 속이기 위해 1로 세팅
```

즉,

- D를 학습할 때는 fake=0
- G를 학습할 때는 fake도 1로 만들어야 함

✓ 5. 한 줄 요약

생성자 G는 “가짜를 진짜처럼 보이게 만드는 역할”이며, 오직 판별자를 속이도록 학습되는 네트워크이다.

원하면

- 생성자/판별자 역할의 차이
- G와 D의 loss 흐름을 표로
- MNIST GAN의 full code
- 흐름도 (GAN training step-by-step 그림)

다 그려줄게!



$\mathbb{E}_{z \sim p(z)}$ 도 알려줘

좋아, $\mathbb{E}_{z \sim p(z)}$ 이 표현이 GAN에서 무슨 뜻인지 “진짜 쉽게 → 정식 → 코드” 순으로 완전 명확하게 설명해줄게.

✓ 1. 초간단 설명

$$\mathbb{E}_{z \sim p(z)}$$

= z 라는 랜덤 벡터를 어떤 확률분포 $p(z)$ 에서 뽑아서 평균을 구한다 는 뜻.

즉,

- z 는 “랜덤 노이즈 벡터”
- $p(z)$ 는 “노이즈를 뽑는 분포”
- 대부분 정규분포 $N(0, 1)$ 사용

그래서

$$\mathbb{E}_{z \sim p(z)}[f(z)]$$

= z 를 여러 번 랜덤하게 뽑아서 $f(z)$ 를 여러 번 계산한 평균

✓ 2. GAN에서 이걸 왜 쓰나요?

생성자 G 는 랜덤 노이즈 z 를 넣어서 이미지를 만든다.

$$G(z), z \sim p(z)$$

여기서 $p(z)$ 는 일반적으로 다음 두 개 중 하나:

- $p(z) = \mathcal{N}(0, I) \leftarrow$ 정규분포(가장 많이 사용)
- $\text{uniform}(-1, 1)$

✓ 3. 정식적인 GAN Loss 공식

Discriminator

$$\max_D \mathbb{E}_{x \sim p_{\text{real}}(x)}[\log D(x)] + \mathbb{E}_{z \sim p(z)}[\log(1 - D(G(z)))]$$

Generator

$$\min_G \mathbb{E}_{z \sim p(z)}[\log(1 - D(G(z)))]$$

혹은 더 잘 학습되는 버전:

$$\max_G \mathbb{E}_{z \sim p(z)} [\log(D(G(z)))]$$

모두 “z를 $p(z)$ 에서 랜덤하게 뽑아서 평균을 구한다”는 의미.

✓ 4. 완전 직관적 접근

- 주사위를 던져서 z를 뽑는 행위 $\rightarrow “z \sim p(z)”$
- 이렇게 뽑힌 z로 이미지 생성 $\rightarrow “G(z)”$
- 이걸 여러 번 반복하여 평균 내기 $\rightarrow “E(…”$

즉,
랜덤 노이즈를 여러 개 뽑아서 그에 대한 loss 평균

✓ 5. 코드에서는 어떻게 구현되나요?

GAN 논문에서 멋지게 적혀 있는

$$z \sim p(z)$$

이걸 PyTorch 코드로 쓰면 그냥:

```
python

z = torch.randn(batch_size, 100) # N(0,1) 정규분포
```

그리고 "기대값 E"는 자동으로 미니배치 평균으로 계산됨:

```
python

loss = criterion(D(G(z)), torch.ones(batch_size, 1))
# 내부적으로 batch 전체의 평균(E) 계산됨
```

즉,

- `randn()` $\rightarrow p(z)$ 에서 샘플링
- `criterion.mean()` $\rightarrow$ 기대값 계산

✓ 6. 한 줄 요약

$$\mathbb{E}_{z \sim p(z)}$$

= 정규분포에서 랜덤 노이즈 z를 뽑아서 그 값들에 대한 평균을 구한다는 것.
GAN의 생성자는 이 랜덤 노이즈를 입력으로 받아 "진짜같은 가짜"를 만든다.

필요하면 아래 것도 설명해줄 수 있어:

- $p(z) = N(0, I)$ 사용하는 이유
- uniform vs normal 비교
- 기대값을 코드에서 수학적으로 정확히 어떻게 구현하는지
- GAN 논문식 공식 → 파이토치식 코드 1:1 대응표

말하면 바로 만들어줄게!



판별자의 목적함수

GAN에서 **판별자(Discriminator, D)**의 목적함수는 진짜는 **1**, 가짜는 **0**으로 잘 맞추는 것, 즉 **이진분류(Binary Classification)** 목적 그 자체이다.

아주 쉽게 → 정석 → 코드 순으로 완전 정리해줄게.

✓ 1. 초간단 요약

판별자의 목표:

$$D(\text{real}) = 1, D(\text{fake}) = 0$$

(진짜는 진짜라고, 가짜는 가짜라고 판단하는 모델)

그래서 판별자의 loss:

$$\text{loss}_D = BCE(D(\text{real}), 1) + BCE(D(\text{fake}), 0)$$

✓ 2. 쉬운 버전 (직관)

- D는 경비원
- real 이미지: “진짜 방문손님” → 문 열어줘야 함(=1)
- fake 이미지: “도둑(생성자 만든 데이터)” → 내쫓아야 함(=0)

즉, 판별자의 목적함수는:

진짜는 진짜로 잘 분류하고, 가짜는 가짜로 잘 분류하도록 G와 경쟁하며 업데이트되는 함수

✓ 3. 정석 수식 (GAN 논문 버전)

판별자 목적함수:

$$\max_D \mathbb{E}_{x \sim p_{\text{real}}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} [\log(1 - D(G(z)))]$$

각 항의 의미:

- $\log D(x)$
→ 진짜 이미지를 **진짜(1)** 라고 할 확률을 높임
- $\log(1 - D(G(z)))$
→ 가짜 이미지를 **가짜(0)** 라고 할 확률을 높임

✓ 4. 왜 "max"인가?

판별자는:

- 진짜 → 1
- 가짜 → 0

이 확률을 높이고 싶다 = log값의 합을 **극대화**해야 한다.

하지만 실전에서는 대부분 **loss minimization**을 쓰므로:

$$\text{loss}_D = -[\log D(x) + \log(1 - D(G(z)))]$$

이걸 **최소화(minimize)** 한다.

✓ 5. PyTorch 코드 1:1 대응

GAN 논문 공식 → 실제 PyTorch 코드

논문 수식

$$\max_D \log D(x) + \log(1 - D(G(z)))$$

PyTorch 코드

```
python

pred_real = D(real)
pred_fake = D(fake.detach())

loss_real = criterion(pred_real, torch.ones(batch, 1))
loss_fake = criterion(pred_fake, torch.zeros(batch, 1))

loss_D = loss_real + loss_fake
loss_D.backward()
optimizer_D.step()
```

- ✓ `criterion = nn.BCELoss()`
- ✓ `ones` = real 이미지 라벨
- ✓ `zeros` = fake 이미지 라벨

✓ 6. 더 깊은 버전 (이게 핵심)

왜 이런 목적함수를 쓰는가?

→ GAN은 **Minimax 게임** 구조이기 때문.

판별자는 다음 게임에서 "승리"하는 것을 목표로 한다.

$$\max_D V(G, D)$$

여기서

$$V(G, D) = \mathbb{E}_{x \sim p_{\text{real}}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} [\log(1 - D(G(z)))]$$

즉,

- D는 이 값을 **최대화**하려고 하고
- G는 이 값을 **최소화**하려고 한다

✓ 7. 한 줄 요약

판별자의 목적함수는 진짜=1, 가짜=0을 정확히 분류하도록 **log-likelihood**를 최대화하는 것.
코드에서는 BCE(real=1) + BCE(fake=0)로 구현된다.

원하면

- 생성자 목적함수 vs 판별자 목적함수 비교 표
- Loss 흐름 흐름도
- GAN 전체 학습 루프 그림
바로 만들어줄게!



$\max_D \mathbb{E}_{x \sim p_{\text{real}}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} [\log(1 - D(G(z)))]$ 에서 $D(G(z))$ 가 1일때, 0일때를 구별해서 계산식, 설명

아주 좋다.

지금 궁금한 건 판별자 목적함수에서 $D(G(z))$ 가 1일 때 vs 0일 때, 각각 loss가 어떻게 되는가? 왜 그런가?
이거지?

완전 명확하게 수식 → 값 대입 → 직관 → 그래프 감각까지 설명해줄게.

✓ 전체 판별자 목적함수

$$\max_D \underbrace{\mathbb{E}_{x \sim p_{\text{real}}} [\log D(x)]}_{\text{진짜} = 1} + \underbrace{\mathbb{E}_{z \sim p(z)} [\log(1 - D(G(z)))]}_{\text{가짜} = 0}$$

여기서 우리가 집중할 항:

$$\log(1 - D(G(z)))$$

여기서 $D(G(z))$ 는 가짜이미지를 판별자 D가 “진짜일 확률”이라고 본 값.

🔥 1. $D(G(z)) = 1$ 일 때 (판별자가 가짜를 진짜로 착각함)

$D(G(z)) = 1$ 이면,

$$\log(1 - D(G(z))) = \log(1 - 1) = \log(0)$$

$\log(0) \rightarrow -\infty$ (마이너스 무한대)

즉,

- Loss = $-\infty$ (엄청 큰 패널티)
- 판별자는 **최악의 실수를 한 것** (가짜를 진짜로 오판)

👉 그래서 D는 이 상황을 피하기 위해

가짜 → **0**으로 만들려고 미친 듯이 학습한다.

🔥 2. $D(G(z)) = 0$ 일 때 (판별자가 가짜를 정확히 가짜로 판단함)

$D(G(z)) = 0$ 이면,

$$\log(1 - 0) = \log(1) = 0$$

loss = 0

즉, 좋은 상태.

- D가 가짜를 0으로 정확히 맞춤
- 벌점 없음
- 판별자 입장에선 완벽한 정답

🔥 가치 표로 보기

$D(G(z))$	$1 - D(G(z))$	$\log(1 - D(G(z)))$	의미
0	1	0	완벽한 정답
0.5	0.5	negative value (≈ -0.693)	반은 맞고 반은 틀림
0.9	0.1	-2.302	거의 틀림
1	0	$-\infty$	최악의 오판

🔥 3. 직관적 의미

판별자 목적함수는
가짜를 진짜라고 할수록 큰 벌점
가짜를 0으로 만들수록 좋은 점수

즉,

- $D(G(z)) \rightarrow 1$ 이면 D가 망한 것
- $D(G(z)) \rightarrow 0$ 이면 D가 성공한 것

🔥 4. 왜 GAN이 안정적으로 학습되는가?

판별자 입장(마지막 항):

$$\log(1 - D(G(z)))$$

이 함수는:

- 왼쪽 끝(0 근처)에서 $\rightarrow \log(1) = 0$
- 오른쪽 끝(1 근처)에서 $\rightarrow \log(0) = -\infty$

그래서 D는 **절대 가짜를 1로 만들지 않으려 함**
즉, "가짜를 0으로 학습시키는 강력한 힘"이 생긴.

🔥 5. 그림 느낌

- x축: $D(G(z))$
- y축: $\log(1 - D(G(z)))$

곡선은 $x=1$ 에 가까울수록 아래로 $\rightarrow$ **-무한대로 급하강**
 $x=0$ 일 때 0

즉, D는 가짜를 1로 만들면 즉시 처맞는다(엄청 큰 패널티).

🔥 6. 한 줄 요약

✓ $D(G(z)) = 1$

$\rightarrow \log(1 - 1) = \log(0) = -\infty$

$\rightarrow$ 판별자에게 엄청난 벌점

✓ $D(G(z)) = 0$

$\rightarrow \log(1) = 0$

$\rightarrow$ 판별자에게 보상(정답)

원하면 다음도 만들어줄게:

- BCE loss와 $\log(1-D(G(z)))$ 의 관계
- 생성자/판별자 loss 그래프
- 논문 수식 → PyTorch 코드 완전 1:1 대응표
- 직관적 설명 그림(곡선 그래프)

바로 만들어줄게!



판별자. 이 공식을 샘플숫자 몇개 제시해서 표로 단 $\log D(x)$ 값도 제시해줘

$D_{\max} \mathbb{E}_{x \sim p_{\text{real}}}[\log D(x)] + \mathbb{E}_{z \sim p(z)}[\log(1 - D(G(z)))]$

좋아,
판별자의 목적함수를 “샘플 몇 개를 실제 수치로 넣어서”
 $\log D(x)$, $\log(1 - D(G(z)))$, 그리고 두 항의 합까지
표로 명확하게 보여줄게.

✓ 1. 판별자 목적함수 (정리)

$$\max_D \underbrace{\mathbb{E}_{x \sim p_{\text{real}}}[\log D(x)]}_{\text{진짜 샘플 항}} + \underbrace{\mathbb{E}_{z \sim p(z)}[\log(1 - D(G(z)))]}_{\text{가짜 샘플 항}}$$

여기서는 평균(E)을 위해
진짜 3개, 가짜 3개 샘플을 가정해서 실제 숫자를 넣어보자.

✓ 2. 가정값

✓ 진짜(real) 이미지에 대한 D 출력

샘플	D(x) (진짜일 확률)
real 1	0.90
real 2	0.70
real 3	0.40

✓ 가짜(fake) 이미지에 대한 D 출력

샘플	D(G(z)) (진짜라고 착각한 확률)
fake 1	0.10
fake 2	0.30

샘플	D(G(z)) (진짜라고 착각한 확률)
fake 3	0.95

⚡ 3. log 값을 직접 계산한 표

(log는 자연로그 ln 기준)

◆ 3-1. 진짜 항: $\log D(x)$

샘플	D(x)	log D(x)
real 1	0.90	-0.1053
real 2	0.70	-0.3567
real 3	0.40	-0.9163

✓ 평균

$$\frac{-0.1053 - 0.3567 - 0.9163}{3} = -0.4594$$

◆ 3-2. 가짜 항: $\log(1 - D(G(z)))$

샘플	D(G(z))	1 - D(G(z))	log(1 - D(G(z)))
fake 1	0.10	0.90	-0.1053
fake 2	0.30	0.70	-0.3567
fake 3	0.95	0.05	-2.9957 (가짜를 진짜로 봄 → 큰 벌점)

✓ 평균

$$\frac{-0.1053 - 0.3567 - 2.9957}{3} = -1.1526$$

⚡ 4. 전체 판별자 목적함수 값

$$V(D) = (\text{real 평균}) + (\text{fake 평균})$$

$$V(D) = (-0.4594) + (-1.1526) = -1.612$$

D는 이 값을 최대화하고 싶음

즉 더 큰(덜 음수) 값이 되도록 학습함.

★ 최종 표 전체 정리

✓ ### 판별자 목적함수를 샘플별로 수치로 계산한 표

◆ (1) 진짜 샘플

샘플	$D(x)$	$\log D(x)$
real 1	0.90	-0.1053
real 2	0.70	-0.3567
real 3	0.40	-0.9163
평균	—	-0.4594

◆ (2) 가짜 샘플

샘플	$D(G(z))$	$1 - D(G(z))$	$\log(1 - D(G(z)))$
fake 1	0.10	0.90	-0.1053
fake 2	0.30	0.70	-0.3567
fake 3	0.95	0.05	-2.9957
평균	—	—	-1.1526

◆ (3) 전체 목적함수

항	평균 로그값
진짜 항 $E[\log D(x)]$	-0.4594
가짜 항 $E[\log(1 - D(G(z)))]$	-1.1526
전체 $V(D)$	-1.612

🔥 직관 요약

- **real**에서 $\log D(x) \rightarrow$ 진짜일 확률이 낮으면 패널티
- **fake**에서 $\log(1 - D(G(z))) \rightarrow$ 가짜를 진짜로 착각할수록 매우 큰 패널티
- fake 3번(0.95)처럼 가짜를 거의 1로 보면 **log 값 = -무한대에 가까움**

필요하면 다음도 만들어줄게:

- 생성자 목적함수도 동일 방식으로 수치 넣어 표 만들기

- BCE Loss 버전으로 다시 정리
- log 그래프와 해석(직관 강조)
원하는 포맷으로 다시 세팅해줄게!



생성자, 판별자 목적함수, 손실함수를 파이토치에서 GAN 으로 어떻게 표현?

GAN 논문公式(목적함수) → **PyTorch 코드(BCE Loss)** 로

“완전히 1:1 대응되게” 정리해줄게.

헷갈리는 부분을 짚 정리해버릴 핵심 구조야.

전체 구조 요약

✓ 논문 목적함수(수식)

판별자(D)

$$\max_D \mathbb{E}_{x \sim p_{\text{real}}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} [\log(1 - D(G(z)))]$$

생성자(G)

$$\min_G \mathbb{E}_{z \sim p(z)} [\log(1 - D(G(z)))]$$

또는 잘 학습되는 버전:

$$\max_G \mathbb{E}_z [\log D(G(z))].$$

PyTorch에서는 어떻게 구현되는가?

PyTorch에서는 **BCELoss (Binary Cross Entropy)** 를 이용해
이 수식들을 그대로 “코드 형태”로 구현한다.

★ 1. 판별자 목적함수 (D)

판별자 D 목표:

- real → 1
- fake → 0

✓ 수식 → PyTorch 대응

수식 부분

$$\log D(x)$$

→ `BCE(D(real), 1)`

$$\log(1 - D(G(z)))$$

→ `BCE(D(fake), 0)`

✓ PyTorch 코드

```
python

criterion = nn.BCELoss()

# ----- 1) Discriminator -----
# real
pred_real = D(real)
loss_real = criterion(pred_real, torch.ones_like(pred_real))

# fake
fake = G(z).detach()    # D 학습 시 G는 고정
pred_fake = D(fake)
loss_fake = criterion(pred_fake, torch.zeros_like(pred_fake))

# 최종 D loss
loss_D = loss_real + loss_fake

loss_D.backward()
optimizer_D.step()
```

★ 2. 생성자 목적함수 (G)

생성자의 목표:

$D(G(z))$ 를 1로 만들기 (판별자를 속이기)

즉,

- 논문 수식:
 $\log(1 - D(G(z)))$ 를 최소화
- 실전 구현:
 $\log(D(G(z)))$ 를 최대화 (학습이 더 안정적)

PyTorch에서는 보통 둘 중 아래 방식 사용함:

✓ 실전 GAN에서 사용하는 생성자 Loss

$$\text{loss}_G = \text{BCE}(D(G(z)), 1)$$

(= “가짜도 진짜처럼 보이게 해라”)

✓ PyTorch 코드

```
python

# ----- 2) Generator -----
fake = G(z)                                # detach 쓰지 않음 (G 학습되어야 함)
pred_fake_for_G = D(fake)

# G는 fake를 진짜(1)로 만들고 싶다
loss_G = criterion(pred_fake_for_G, torch.ones_like(pred_fake_for_G))

loss_G.backward()
optimizer_G.step()
```

★ 3. 전체 구조 표로 1:1 대응시키기

역할	논문 수식	PyTorch BCE 표현
D - real	$\log D(x)$	<code>BCE(D(real), 1)</code>
D - fake	$\log(1 - D(G(z)))$	<code>BCE(D(fake), 0)</code>
D 전체	최대화	<code>loss_D = loss_real + loss_fake</code>
G 기본	$\log(1 - D(G(z)))$ 최소화	<code>BCE(D(fake), 1)</code>
G 대체(실전)	$\log(D(G(z)))$ 최대화	동일하게 <code>BCE(D(fake), 1)</code>

즉, PyTorch에서는 **BCE Loss**가 논문식 **log-loss**와 동일한 역할을 수행한다.

★ 4. 정리: PyTorch GAN 학습 loop 완전판

```
python

criterion = nn.BCELoss()

for real, _ in dataloader:
    batch = real.size(0)
    real = real.to(device)

    # -----
    # 1) Train Discriminator
    # -----
    z = torch.randn(batch, 100, device=device)
    fake = G(z)

    pred_real = D(real)
    pred_fake = D(fake.detach())

    loss_real = criterion(pred_real, torch.ones_like(pred_real)) # real → 1
    loss_fake = criterion(pred_fake, torch.zeros_like(pred_fake)) # fake → 0
    loss_D = loss_real + loss_fake
```

```
optimizer_D.zero_grad()
loss_D.backward()
optimizer_D.step()

# -----
# 2) Train Generator
# -----
z = torch.randn(batch, 100, device=device)
fake = G(z)
pred_fake_for_G = D(fake)

loss_G = criterion(pred_fake_for_G, torch.ones_like(pred_fake_for_G)) # fake → 1

optimizer_G.zero_grad()
loss_G.backward()
optimizer_G.step()
```

최종 요약

판별자 D

$$\log D(x) + \log(1 - D(G(z)))$$

→ PyTorch

```
loss_D = BCE(D(real), 1) + BCE(D(fake), 0)
```

생성자 G

$$\log(1 - D(G(z))) \text{ (최소화) 또는 } \log(D(G(z))) \text{ (최대화)}$$

→ PyTorch

```
loss_G = BCE(D(fake), 1)
```

원하면:

- GAN 논문 공식 ↔ PyTorch 전체 대응표(1장 PDF 스타일)
- DCGAN 코드 버전
- 수식과 그래프 포함 초보자용 정리이미지
- WGAN/WGAN-GP까지 확장

도 만들어줄게!



손글씨 mnist로 모델은 최소화

좋아!

손글씨 MNIST + GAN + 최소 모델 구조

즉, 가능한 가장 단순한 Generator/Discriminator + 풀코드 전체 + 설명까지
“초간단 GAN(MNIST)” 최저 사양 버전을 제공해줄게.

- ✓ 모델 최소화
- ✓ 코드 짧게
- ✓ GAN 원리 그대로
- ✓ CPU에서도 바로 학습 가능
- ✓ MNIST 28×28 흑백 기준

1. 전체 구조(초간단)

- **Generator:** MLP 3층
- **Discriminator:** MLP 3층
- **데이터:** MNIST 28×28 → 784차원
- **Noise:** $z = 100$ 차원

최소 모델이라 CNN 없이 Fully Connected만 사용.

2. 완성 코드 (MNIST 초간단 GAN 전체 구조)

```
python

import torch
import torch.nn as nn
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

# =====
# 1. Hyperparameters
# =====
batch_size = 64
z_dim = 100
lr = 0.0002
epochs = 20
device = "cuda" if torch.cuda.is_available() else "cpu"

# =====
# 2. Data (MNIST)
# =====
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,)) # -1 ~ 1 범위
])

dataset = datasets.MNIST(root="./data", train=True,
                        transform=transform, download=True)
loader = DataLoader(dataset, batch_size=batch_size, shuffle=True)

# =====
# 3. Generator (최소 모델)
# =====
class Generator(nn.Module):
    def __init__(self):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(z_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 784),
```

```

        nn.Tanh() # MNIST -1~1 맞춤
    )

    def forward(self, z):
        img = self.model(z)
        return img.view(-1, 1, 28, 28)

# =====
# 4. Discriminator (최소 모델)
# =====
class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(784, 128),
            nn.LeakyReLU(0.2),
            nn.Linear(128, 1),
            nn.Sigmoid() # 진짜 확률
        )

    def forward(self, x):
        x = x.view(-1, 784)
        return self.model(x)

G = Generator().to(device)
D = Discriminator().to(device)

criterion = nn.BCELoss()
opt_G = torch.optim.Adam(G.parameters(), lr=lr)
opt_D = torch.optim.Adam(D.parameters(), lr=lr)

# =====
# 5. Training
# =====
for epoch in range(epochs):
    for real, _ in loader:
        real = real.to(device)
        batch = real.size(0)

        # -----
        # 1) Train Discriminator
        # -----
        z = torch.randn(batch, z_dim, device=device)
        fake = G(z).detach()

        pred_real = D(real)
        pred_fake = D(fake)

        loss_real = criterion(pred_real, torch.ones_like(pred_real))
        loss_fake = criterion(pred_fake, torch.zeros_like(pred_fake))
        loss_D = loss_real + loss_fake

        opt_D.zero_grad()
        loss_D.backward()
        opt_D.step()

        # -----
        # 2) Train Generator
        # -----
        z = torch.randn(batch, z_dim, device=device)
        fake = G(z)
        pred_fake_G = D(fake)

        loss_G = criterion(pred_fake_G, torch.ones_like(pred_fake_G))

        opt_G.zero_grad()
        loss_G.backward()
        opt_G.step()

    print(f"[Epoch {epoch+1}/{epochs}] loss_D={loss_D.item():.4f} loss_G={loss_G.item():.4f}")

print("Training complete!")

```

🎯 3. 최소 모델 구조 한눈에 보기

✓ Generator ($z=100 \rightarrow 28 \times 28$)

SCSS

```
Linear(100 → 128)
ReLU
Linear(128 → 784)
Tanh
```

✓ Discriminator ($28 \times 28 \rightarrow 1$)

SCSS

```
Linear(784 → 128)
LeakyReLU(0.2)
Linear(128 → 1)
Sigmoid
```

🎯 4. 결과 이미지 시각화 추가 (선택)

python

```
import matplotlib.pyplot as plt

z = torch.randn(16, z_dim, device=device)
fake = G(z).detach().cpu()

plt.figure(figsize=(4,4))
for i in range(16):
    plt.subplot(4,4,i+1)
    plt.imshow(fake[i][0], cmap="gray")
    plt.axis("off")
plt.show()
```

🎯 5. 초간단 작동 원리 요약

- **D**: $\text{real} \rightarrow 1$, $\text{fake} \rightarrow 0$ 맞추는 2-class classifier
- **G**: $D(\text{fake})$ 를 1로 속이도록 학습
- **BCE Loss** 그대로 GAN 논문 공식과 동일

🔥 원하면 더 만들어줄게!

- CNN 기반 DCGAN 최소 버전
- 메모리 최적화 버전
- 학습곡선(평균 loss) 시각화

- 생성자/판별자 한 장으로 구조도

필요한 방향 알려줘!



완성 코드 (MNIST 초간단 GAN 전체 구조) 를 아티클별로 나누어서, 자세히 설명

바로 그 “완성 코드”를 조각(아티클) 별로 잘라서
코드 → 역할 → 수식과 연결까지 차근차근 뜯어볼게요.
 (코드블럭은 안 쓰고, 그냥 텍스트로 적을게요!)

0. 전체 구조 한 번만 다시 상기

우리가 쓸 GAN 구성은:

- 데이터: MNIST (28×28 손글씨)
- G(Generator): MLP 2층 ($z=100 \rightarrow 784 \rightarrow 28 \times 28$)
- D(Discriminator): MLP 2층 ($784 \rightarrow 128 \rightarrow 1$)
- Loss: BCELoss (논문에서의 $\log D(x)$, $\log(1-D(G(z)))$ 를 대신)

이걸 아티클별로 나눠서 설명합니다.

1. 라이브러리 & 하이퍼파라미터 설정

[코드]

```
import torch
import torch.nn as nn
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

batch_size = 64
z_dim = 100
lr = 0.0002
epochs = 20
device = "cuda" if torch.cuda.is_available() else "cpu"
```

[설명]

1. `import torch, import torch.nn as nn`

- PyTorch 기본, 신경망 층(Linear, Conv 등)을 쓰려고 nn 모듈 импорт.

2. `from torchvision import datasets, transforms`

- torchvision은 이미지 데이터셋+전처리 모음.
- 여기서는 `datasets.MNIST`와 `transforms`를 사용.

3. `from torch.utils.data import DataLoader`

- `DataLoader`: 배치 단위로 데이터를 제공.

4. `batch_size = 64`

- 한 번에 64개 이미지씩 학습.
- 미니배치 기대값 $\approx$ 수식에서의 E(기대값) 역할.

5. `z_dim = 100`

- 노이즈 벡터 z 의 차원.
- 논문에서 $z \sim p(z)$ 의 z 가 여기서는 `torch.randn(batch, 100)`.

6. `lr = 0.0002`

- learning rate. Adam 최적화에서 쓰는 기본 값.

7. `epochs = 20`

- 전체 데이터셋을 20번 반복 학습.

8. `device = "cuda" if torch.cuda.is_available() else "cpu"`

- GPU가 있으면 GPU 사용, 없으면 CPU.

2. MNIST 데이터셋 & 전처리

[코드]

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,)) # -1 ~ 1 범위
])

dataset = datasets.MNIST(root="./data", train=True,
    transform=transform, download=True)

loader = DataLoader(dataset, batch_size=batch_size, shuffle=True)
```

[설명]

1. `transforms.ToTensor()`

- PIL 이미지(0~255) $\rightarrow$ 0~1 float Tensor (C,H,W).

2. `transforms.Normalize((0.5,), (0.5,))`

- $(x - 0.5) / 0.5 \rightarrow$ 입력을 -1 ~ 1 근처로 맞춤.
- Generator 출력이 `Tanh()`(-1~1)을 쓰기 때문에, real 이미지도 같은 범위로 맞춰주는 것.

3. `datasets.MNIST`

- `root="./data"`: 데이터를 저장할 폴더.
- `train=True`: 학습용 train 데이터 사용.

- **transform=transform**: 위에서 정의한 전처리 적용.
- **download=True**: 없다면 자동 다운로드.

4. DataLoader(dataset, batch_size=batch_size, shuffle=True)

- 각 반복마다 (이미지, 라벨) batch를 뽑아줌.
- GAN에서 라벨(0~9)은 안 쓰고, **이미지(real)** 만 사용.

3. Generator 최소 모델

[코드]

```
class Generator(nn.Module):
    def init(self):
        super().init()
        self.model = nn.Sequential(
            nn.Linear(z_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 784),
            nn.Tanh() # MNIST -1~1 맞춤
        )
```

ruby

```
def forward(self, z):
    img = self.model(z)
    return img.view(-1, 1, 28, 28)
```

G = Generator().to(device)

[설명]

1. class Generator(nn.Module):

- PyTorch 모델은 **nn.Module** 상속.

2. nn.Linear(z_dim, 128)

- 입력: 랜덤 노이즈 z (크기 100)
- 출력: 128차원 은닉층
- 수식으로는:
 $h = \text{ReLU}(W_1 z + b_1)$

3. nn.ReLU()

- 비선형 활성화 함수.
- 음수는 0, 양수는 그대로.

4. nn.Linear(128, 784)

- $128 \rightarrow 784$ 로 매핑.
- $784 = 28 \times 28$, 즉 한 장의 이미지 픽셀 수.

5. nn.Tanh()

- 출력 범위를 -1 ~ 1 로 제한.
- 이 값이 나중에 real 이미지(-1~1로 normalize 된 것)과 비교되며 D 입력이 됨.

6. forward()

- 입력: `z (batch_size, z_dim)`
- `img = self.model(z) → (batch_size, 784)`
- `view(-1, 1, 28, 28)`
 - 784를 1×28×28 이미지로 reshape.

7. 수식 연결

- Generator는

$$G(z) = \text{Tanh}(W2 \text{ReLU}(W1 z + b1) + b2)$$
- 이게 논문에서 $D(G(z))$ 의 그 G.

4. Discriminator 최소 모델

[코드]

```
class Discriminator(nn.Module):
    def init(self):
        super().init()
        self.model = nn.Sequential(
            nn.Linear(784, 128),
            nn.LeakyReLU(0.2),
            nn.Linear(128, 1),
            nn.Sigmoid() # 진짜 확률
        )
```

ruby

```
def forward(self, x):
    x = x.view(-1, 784)
    return self.model(x)
```

`D = Discriminator().to(device)`

[설명]

1. `nn.Linear(784, 128)`

- 28×28 이미지 → 784를 한 줄로 펼친 후 128차원으로 축소.

2. `nn.LeakyReLU(0.2)`

- ReLU의 변형: 음수도 0으로 완전히 죽이지 않고 0.2x로 유지.
- GAN에서 D의 gradient 죽는 문제를 완화하는 전통적인 선택.

3. `nn.Linear(128, 1)`

- 최종 출력 1차원: “이 이미지가 진짜일 확률”.

4. `nn.Sigmoid()`

- 0~1 사이 값.
- $D(x) \Rightarrow$ “진짜같다”는 확률로 해석.

5. forward()

- $x.view(-1, 784) : (B, 1, 28, 28) \rightarrow (B, 784)$
- `self.model(x)` 결과: (B, 1)

6. 수식 연결

- $D(x) = \text{Sigmoid}(W4 \text{ LeakyReLU}(W3 x + b3) + b4)$
- 논문에서의 $D(x)$, $D(G(z))$ 가 이 함수.

5. 손실함수 & 옵티마이저 (GAN 목적함수와 연결)

[코드]

```
criterion = nn.BCELoss()
opt_G = torch.optim.Adam(G.parameters(), lr=lr)
opt_D = torch.optim.Adam(D.parameters(), lr=lr)
```

[설명]

1. nn.BCELoss()

- Binary Cross Entropy:

$$\text{BCE}(p, y) = - [y \log p + (1-y) \log(1-p)]$$
- 여기서 $p = D(x)$, y 는 1(진짜), 0(가짜).

2. 논문과의 연결

- 논문에서 판별자 목적함수:

$$E_{x \sim p_{\text{real}}} [\log D(x)] + E_{z \sim p(z)} [\log(1 - D(G(z)))]$$
- BCE 관점에서 보면:
 - real 항: $y=1 \rightarrow -\text{BCE}(D(\text{real}), 1)$ 에서 $-\log D(x)$ 에 해당
 - fake 항: $y=0 \rightarrow -\text{BCE}(D(\text{fake}), 0)$ 에서 $-\log(1-D(G(z)))$ 에 해당
- 우리는 **loss를 최소화**하므로

$$\text{loss}_D = \text{BCE}(D(\text{real}), 1) + \text{BCE}(D(\text{fake}), 0)$$
 를 최소화 ==

$$-[\log D(x) + \log(1-D(G(z)))]$$
를 최소화 ==

$$\log D(x) + \log(1-D(G(z)))$$
를 최대화.

3. Adam 옵티마이저

- `opt_G`: 생성자 파라미터 업데이트용.
- `opt_D`: 판별자 파라미터 업데이트용.

6. 학습 루프 - Discriminator 부분

[코드]

```
for epoch in range(epochs):
for real, _ in loader:
real = real.to(device)
batch = real.size(0)
```

markdown

```
# 1) Train Discriminator
z = torch.randn(batch, z_dim, device=device)
fake = G(z).detach()

pred_real = D(real)
pred_fake = D(fake)

loss_real = criterion(pred_real, torch.ones_like(pred_real))
loss_fake = criterion(pred_fake, torch.zeros_like(pred_fake))
loss_D = loss_real + loss_fake

opt_D.zero_grad()
loss_D.backward()
opt_D.step()
```

[설명]

1. for real, _ in loader:

- real: (B, 1, 28, 28) 손글씨 이미지.
- 라벨 _은 여기서 사용 안 함.

2. z = torch.randn(batch, z_dim, device=device)

- $z \sim N(0, I)$: 논문에서의 $p(z)$.
- 각 샘플마다 100차원 노이즈 벡터.

3. fake = G(z).detach()

- G가 만든 가짜 이미지.
- detach()는 D만 학습하도록 gradient 끊기.
- 지금은 "판별자 단계"이므로 G는 고정.

4. pred_real = D(real)

- D(real): 진짜 이미지에 대한 D의 출력 (진짜 확률).

5. pred_fake = D(fake)

- D(fake): 가짜에 대한 D의 출력 (진짜라고 착각하는 정도).

6. loss_real = criterion(pred_real, torch.ones_like(pred_real))

- BCE(D(real), 1)
- 수식: $-\log D(x)$ 를 평균 형태로 계산.

7. loss_fake = criterion(pred_fake, torch.zeros_like(pred_fake))

- BCE(D(fake), 0)
- 수식: $-\log(1 - D(G(z)))$.

8. `loss_D = loss_real + loss_fake`

- 논문 수식:

$$\max_D [E \log D(x) + E \log(1-D(G(z)))]$$
- 코드에서는 이걸 음수 형태 BCE로 최소화 하는 방식.

9. `opt_D.zero_grad(), loss_D.backward(), opt_D.step()`

- 기존 gradient 초기화 → 역전파 → 파라미터 업데이트.

7. 학습 루프 - Generator 부분

[코드]

python

```
# 2) Train Generator
z = torch.randn(batch, z_dim, device=device)
fake = G(z)
pred_fake_G = D(fake)

loss_G = criterion(pred_fake_G, torch.ones_like(pred_fake_G))

opt_G.zero_grad()
loss_G.backward()
opt_G.step()

print(f"[Epoch {epoch+1}/{epochs}] loss_D={loss_D.item():.4f} loss_G={loss_G.item():.4f}")
```

`print("Training complete!")`

[설명]

1. 다시 z 생성

- `z = torch.randn(batch, z_dim, device=device)`

2. `fake = G(z)`

- 여기서는 `detach()` 안 씀.
- 생성자 파라미터에 gradient가 흘러가야 함.

3. `pred_fake_G = D(fake)`

- D가 본 G의 가짜 이미지에 대한 “진짜일 확률”.

4. `loss_G = criterion(pred_fake_G, torch.ones_like(pred_fake_G))`

- $BCE(D(G(z)), 1)$
- 생성자의 목표:
 → $D(G(z))$ 를 1에 가깝게 만들기.
 → 논문에서의 $\max_G E[\log D(G(z))]$ 와 같은 효과.
- 수식으로 보면:
 - $BCE(D(G(z)), 1) = -\log D(G(z))$
 - 이걸 최소화 $\Leftrightarrow \log D(G(z))$ 를 최대화.

5. `opt_G.zero_grad(), loss_G.backward(), opt_G.step()`

- 생성자 파라미터만 업데이트.

6. `print(...)`

- 마지막 배치 기준으로 D와 G의 loss 출력.

8. 수식 ↔ 코드 최종 대응표

역할	논문 수식	PyTorch 코드
D - real	$E_x[\log D(x)]$	<code>loss_real = BCE(D(real), 1)</code>
D - fake	$E_z[\log(1 - D(G(z)))]$	<code>loss_fake = BCE(D(fake), 0)</code>
D 전체	$\max_D (... + ...)$	<code>loss_D = loss_real + loss_fake (minimize)</code>
G	$\max_G E_z[\log D(G(z))]$	<code>loss_G = BCE(D(G(z)), 1)</code>

← 정리 한 줄

- 이 코드는 가장 단순한 형태의 **MNIST GAN**이고,
- 논문에 나온 목적함수
 $E_x[\log D(x)] + E_z[\log(1 - D(G(z)))] / E_z[\log D(G(z))]$
 를 **PyTorch**의 **BCELoss**로 구현한 것입니다.

원하면 다음도 해줄 수 있어요:

- 이 minimal GAN을 **CNN 버전(DCGAN식)** 으로 확장
- loss 값과 함께 **에포크별 생성 이미지 변화** 시각화 코드
- 코드 각 줄에 수식/그림 달아서 “교육용 슬라이드 버전”으로 정리

바로 이어서 만들어줄게요.



BCE함수가 알아서 다하는거야?

결론부터 말하면 **그렇다**.

GAN에서 복잡해 보이는

- $\log D(x)$
- $\log(1 - D(G(z)))$
- 생성자/판별자 목적함수

이 모두 BCELoss 안에 그대로 들어있다.

그래서 우리는 BCE만 호출하면 자동으로 논문 수식이 구현된다.

하지만 “BCE가 어떻게 그걸 모두 계산하는가?”

이 부분을 아주 명확하게 정리해줄게.

1. BCE는 어떤 함수인가?

PyTorch의 `nn.BCELoss()`는 아래 수식을 그대로 계산한다.

$$\text{BCE}(p, y) = -(y \log p + (1 - y) \log(1 - p))$$

여기서

- $p = D(x)$ 또는 $D(G(z))$
- $y = 1$ 또는 0 (정답 라벨)

즉, BCE 한 줄이 두 개의 항을 모두 계산한다.

2. 판별자의 loss가 BCE에서 자동으로 계산되는 이유

판별자 목적함수는:

$$\max_D \log D(x) + \log(1 - D(G(z)))$$

PyTorch에서는 loss를 최소화하도록 아래처럼 쓴다:

```
ini
loss_D = BCE(D(real), 1) + BCE(D(fake), 0)
```

이걸 BCE 수식으로 직접 풀어보면:

① BCE(D(real), 1)

$$-\log D(x)$$

(D가 진짜를 진짜로 맞추도록)

② BCE(D(fake), 0)

$$-\log(1 - D(G(z)))$$

(D가 가짜를 0으로 맞추도록)

그래서

```
bash
loss_D = -(log D(x) + log(1 - D(G(z))))
```

loss_D를 최소화한다 =

$\log D(x) + \log(1 - D(G(z)))$ 를 최대화한다.

즉, 논문 수식을 그대로 구현한 것.

→ BCE가 자동으로 $\log$, $\log(1-p)$ 를 모두 계산해주기 때문에 따로 할 필요가 없다.

3. 생성자 loss도 BCE 한 줄이 자동으로 처리함

논문에서는 G의 loss:

$$\min_G \log(1 - D(G(z)))$$

실전에서는 더 안정적인 loss:

$$\max_G \log D(G(z))$$

PyTorch에서는 아래처럼 씀:

```
ini
loss_G = BCE(D(G(z)), 1)
```

이걸 풀면,

$$\text{BCE}(D(G(z)), 1) = -\log(D(G(z)))$$

loss_G를 최소화 → $\log(D(G(z)))$ 최대화

즉, 생성자가 D를 속이는 방향으로 학습됨.

→ 이것도 BCE가 자동으로 $\log$ 계산을 대신해주는 것.

4. 직관적으로 보면 BCE는 "GAN 공식 자동 변환기"

논문 공식

- $\log D(x)$
- $\log(1 - D(G(z)))$
- $y \log p + (1-y)\log(1-p)$
- 생성자 vs 판별자 독립된 목적함수

PyTorch GAN 코드

- $\text{BCE}(D(\text{real}), 1)$
- $\text{BCE}(D(\text{fake}), 0)$
- $\text{BCE}(D(\text{fake}), 1)$

이 모든 것을 **BCE 하나로 표현**할 수 있게 한 것이 PyTorch의 큰 장점.

5. 정말 BCE만 있으면 되는지? → 네

GAN에서 BCE가 하는 일:

역할	BCE가 해주는 것
log	자동
log(1-p)	자동
진짜 vs 가짜 분류	자동
생성자 손실/판별자 손실 분리	자동
미니배치 평균 E[] 계산	자동
역전파	자동
그래디언트 계산	자동

너는 단지

```
scss

criterion(pred, target)
```

이 한 줄만 쓰면 됨.

6. 한 줄 요약

- ✓ BCELoss는 GAN 논문에서 등장하는 모든 $\log$, $\log(1-p)$ 식을 내부에서 자동으로 계산해준다.
- ✓ 그래서 GAN 목적함수 전체를 **BCE 한 줄로 그대로 구현**할 수 있다.

원하면:

- BCE 수식을 ASCII로 직접 시각화
- BCE 없이 직접 $\log$ 수식으로 GAN 구현하기(교육용)
- BCELoss와 BCELossWithLogits 차이
- WGAN에서는 BCE 없어지는 이유

도 알려줄게!



BCELossWithLogits 은?

좋아!

BCELossWithLogits는 GAN을 제대로 이해하려면 반드시 알아야 하는 핵심이야.

특히 **WGAN, StyleGAN, Diffusion** 이전의 대부분 GAN 논문 구현에서 굉장히 중요한 개념.

지금부터 완전 직관 → 공식 → PyTorch 동작 원리 → 왜 필요한가? 까지 가장 명확하게 설명해줄게.

🎯 1. 결론 먼저:

BCELoss + Sigmoid 를 합쳐놓은 버전 = **BCEWithLogitsLoss**

즉,

```
nginx
Sigmoid + BCELoss
```

이 두 개를 한 번에, 더 안정적으로 계산하는 함수.

🎯 2. 왜 합쳤을까? (핵심 문제!)

보통 GAN에서는 아래처럼 D의 마지막 층이 있다:

```
nginx
x → Linear → Sigmoid → BCELoss
```

하지만 **Sigmoid → log → (1-p)** 계산 조합은 수치적으로 불안정함.

예시:

- D(G(z))가 0.00001이면 $\log(0.00001)$ 계산
- or D(G(z))가 1에 가까우면 $\log(1-0.999999)$
- float precision 문제 발생, gradient 사라짐
- 특히 GAN에서 D가 강해지면 생성자 G의 gradient가 사라짐 (vanishing gradient)

그런데 **Sigmoid + BCE** 를 따로 쓰면 이런 문제가 생김.

그래서 PyTorch가 “수학적으로 안전한 방식으로 합쳐서 계산” 하는 게 **BCEWithLogitsLoss**.

🎯 3. BCEWithLogitsLoss가 하는 일

이 함수는 내부적으로 아래 수식을 계산한다:

$$\text{loss} = -(y \cdot \log \sigma(x) + (1 - y) \cdot \log(1 - \sigma(x)))$$

하지만

$\log \sigma(x)$ 와 $\log(1 - \sigma(x))$ 를 직접 계산하지 않고

잘-known numerical stable 형태:

$$\text{loss} = \max(x, 0) - x \cdot y + \log(1 + e^{-|x|})$$

이걸 사용한다.

- Sigmoid 결과를 안 쓰고
- 마지막 Linear 출력(logits)을 바로 받음

그래서 이름이:

- Binary Cross Entropy with **Logits**
- 즉, "Sigmoid 하기 전 값"을 그대로 넣는 BCE.

4. GAN에서 어떻게 쓰는가?

기존 (안전하지 않은 방식)

```
ini

output = torch.sigmoid(D(x))
loss = BCELoss(output, target)
```

BCEWithLogitsLoss 사용하는 방식 (추천)

```
ini

loss = BCEWithLogitsLoss(D(x), target)
```

그리고 **D** 모델에서 마지막 **Sigmoid** 를 제거해야 한다!

즉,

 **D**에 Sigmoid 넣지 않음

 **BCEWithLogitsLoss**가 Sigmoid를 내부에서 대신함

5. 왜 GAN에서 BCEWithLogitsLoss가 더 좋은가?

■ 이유 1: 수치 안정성 극대화

Sigmoid + log 조합보다 100배 안정적이다.

■ 이유 2: gradient vanishing 문제가 크게 줄어들음

- 특히 생성자 G의 학습이 훨씬 안정됨
- D가 너무 잘 학습되었을 때도 완전 죽지 않음

■ 이유 3: GAN 코드가 더 간단해짐

D의 마지막 층에 Sigmoid를 넣을 필요 없음.

🎯 6. 코드 비교 (매우 중요)

❌ 이전 방식 (비추천)

```
scss
class D(nn.Module):
    def __init__(...):
        ...
        self.last = nn.Sigmoid()

loss = BCELoss(D(x), target)
```

✅ 권장 방식 (Sigmoid 제거)

```
scss
class D(nn.Module):
    def __init__(...):
        ...
        # 마지막 Sigmoid 제거

loss = BCEWithLogitsLoss(D(x), target)
```

🎯 7. MNIST GAN에 적용하면?

기존 BCELoss 버전(D에 Sigmoid 있음)

```
ini
pred = D(real)          # D에 Sigmoid 포함됨
loss_real = BCE(pred, ones)
```

BCEWithLogitsLoss 버전(추천)

```
ruby
criterion = nn.BCEWithLogitsLoss()

# D는 아래처럼 정의
class Discriminator(nn.Module):
    def __init__(...):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(784, 128),
            nn.LeakyReLU(0.2),
            nn.Linear(128, 1)
        )
        # Sigmoid 제거
```

```
pred = D(real)    # 이 값은 logit
loss_real = criterion(pred, torch.ones_like(pred))
```

8. 한 줄 요약

BCEWithLogitsLoss = Sigmoid + BCELoss를 합친, 매우 안정적인 버전.

GAN에서 D의 마지막 Sigmoid를 없애고

BCEWithLogitsLoss를 쓰는 것이 권장되는 표준 방식이다.

원하면:

- GAN minimal 코드 전체를 BCEWithLogitsLoss 버전으로 재구성
- Sigmoid vs BCEWithLogitsLoss 그래프 비교
- 왜 Gradient Vanishing이 줄어드는지 수학적으로 분석

도 바로 만들어줄게!